AI - Assisted Coding-Assignment -20.2

HTNO:2303A52228
Batch:37

Lab 20 - Security Testing: Identifying Vulnerabilities in AI-Generated Code.

Task Description-1
Password Security Enhancement..
Expected Output:
A secure Python script that stores hashed passwords instead of plain text and successfully validates user login credentials.
Prompt:
#Generate a Python user registration and login system with the following steps:
#1. First create an insecure version storing passwords in plain text
#2. Identify and explain all security vulnerabilities
#3. Refactor the code using bcrypt hashing
#4. Provide test cases for validation.
CODE AND OUTPUT :

```python
import bcrypt
users = {}
def register():
    username = input("Enter username: ")
    if username in users:
        print("Username already exists.")
        return
    password = input("Enter password: ").encode()
    hashed = bcrypt.hashpw(password, bcrypt.gensalt())
    users[username] = hashed
    print("User registered successfully.")
def login():
    username = input("Enter username: ")
    password = input("Enter password: ").encode()
    if username in users and bcrypt.checkpw(password, users[username]):
        print("Login successful.")
    else:
        print("Invalid username or password.")
while True:
    print("\n1. Register\n2. Login\n3. Exit")
    choice = input("Enter choice: ")
    if choice == '1':
        register()
    elif choice == '2':
        login()
    elif choice == '3':
        break
    else:
        print("Invalid choice")
```

```
1. Register
2. Login
3. Exit
Enter choice: 1
Enter username: Abhi
Enter password: 1234
User registered successfully.

1. Register
User registered successfully.

1. Register

1. Register
1. Register
2. Login
3. Exit
Enter choice: 2
Enter username: Abhi
Enter password: 12345
Invalid username or password.

1. Register
2. Login
3. Exit
Enter choice: 3
```

Explanation:
Your code implements a simple user authentication system with two main features: registration and login. It uses a dictionary to store usernames and passwords (in hashed form for security). During registration, the program checks if the username already exists; if not, it hashes the password (using bcrypt) and stores it. During login, it takes the entered password, hashes it, and compares it with the stored hash to verify the user.

Task Description-2
Secure API Key Management.
Expected Output:
A revised script that securely accesses API keys through environment variables instead of hard coding them.

Prompt:
Generate a Python script that connects to an external API using a hardcoded API key.
1. Identify the security risks of hardcoded API keys
2. Refactor the code to use environment variables instead of hardcoding
3. Add proper error handling
4. Show how to set environment variables on Windows/Linux
5. Provide sample input and output.

CODE AND OUTPUT :

```python
import os
import requests
def get_weather():
    # Get API key from environment variable
    api_key = os.getenv("API_KEY")
    if not api_key:
        print("Error: API key not found. Please set environment variable.")
        return
    city = input("Enter city name: ")
    # API endpoint (key is NOT hardcoded)
    url = f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}"
    try:
        response = requests.get(url)
        data = response.json()
        if response.status_code == 200:
            print("City:", data["name"])
            print("Temperature:", data["main"]["temp"])
            print("Weather:", data["weather"][0]["description"])
        else:
            print("Error:", data.get("message", "Failed to fetch data"))
    except Exception as e:
        print("Error occurred:", e)
get_weather()
```

```
Enter city name: chennai
City: Chennai
Temperature: 305.15
Weather: scattered clouds
```

Explanation:
In the original program, the API key is written directly inside the code, which is risky because anyone who sees the code can misuse the key. This can lead to data leaks or unauthorized access.To make it secure, the API key is removed from the code and stored in an environment variable. The program then reads the key using os.getenv(), which keeps sensitive information hidden.This approach improves security, makes the code safer to share, and follows best practices used in real-world applications.

Task Description-3
Weak Encryption Detection.
Expected Output:
An improved encryption program implementing stronger and more secure

cryptographic practices.

Prompt:

#Generate a Python encryption program using a weak method like Base64.
#Then identify its weaknesses and refactor it using strong encryption (Fernet).
#Provide explanation and test output.
Weak encryption using Base64.

CODE AND OUTPUT:

```python
from cryptography.fernet import Fernet
def generate_key():
    return Fernet.generate_key()
def strong_encrypt(plain_text, key):
    f = Fernet(key)
    encrypted_text = f.encrypt(plain_text.encode("utf-8"))
    return encrypted_text.decode("utf-8")
def strong_decrypt(encrypted_text, key):
    f = Fernet(key)
    decrypted_text = f.decrypt(encrypted_text.encode("utf-8"))
    return decrypted_text.decode("utf-8")
# Test strong encryption
key = generate_key()
encrypted_text = strong_encrypt(plain_text, key)
print("Encrypted (Fernet):", encrypted_text)
decrypted_text = strong_decrypt(encrypted_text, key)
print("Decrypted (Fernet):", decrypted_text)
```

```
Encoded (Base64): VGhpcyBpcyBhIHNlY3JldCBtZXNzYWdlLg==
Decoded (Base64): This is a secret message.
Encrypted (Fernet): gAAAAABpy1ICm6A-mqNts9APKCwVYE8FFCLZCDlYBObxWrtNvzZPpDCAGBhe1aA7qrQ3Wc6G6qVCmMKXMqE6l
_yVEQOl8TJ6R8oXT4quqg=
Decrypted (Fernet): This is a secret message.
```

Explanation:

The program uses the cryptography library's Fernet module to perform strong encryption. First, a secret key is generated using Fernet.generate_key(), which is essential because this key is required for both encrypting and decrypting the data. Then, a cipher object is created using this key, which acts as the tool for encryption and decryption operations. The encrypt() function takes a normal text message, converts it into bytes, and encrypts it using the cipher, producing an unreadable encrypted output. The decrypt() function takes this encrypted data and uses the same key to convert it back into the original readable text.

Task Description-4

SQL to Pandas Query.Real-Time Application: Security Review of AI-Generated

Web Service.

Expected Output:

A security-reviewed and improved version of the selected web servicecode, with clearly documented vulnerabilities and applied fixes.

Prompt:

#Generate a Flask login API with basic functionality.
#1. Identify at least 3 security vulnerabilities
#2. Refactor the code using secure practices (bcrypt, validation)
#3. Provide improved version and comparison.

CODE AND OUTPUT :

```
from flask import Flask, request, jsonify
import bcrypt
app = Flask(__name__)
users = {}
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    if not data or 'username' not in data or 'password' not in data:
        return jsonify({'error': 'Invalid input'}), 400
    username = data['username']
    password = data['password']
    if username in users:
        return jsonify({'error': 'Username already exists'}), 400
    hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
    users[username] = hashed
    return jsonify({'message': 'User registered successfully'}), 201
@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    if not data or 'username' not in data or 'password' not in data:
        return jsonify({'error': 'Invalid input'}), 400
    username = data['username']
    password = data['password'].encode()
    if username in users and bcrypt.checkpw(password, users[username]):
        return jsonify({'message': 'Login successful'}), 200
    else:
        return jsonify({'error': 'Invalid credentials'}), 401
@app.route('/')
```

Explanation:
This code is a Flask API for user registration and login. It stores usernames in a dictionary and secures passwords using bcrypt hashing instead of plain text. During registration, it hashes and saves the password, and during login, it checks the entered password against the stored hash. The API works through POST requests (/register, /login) and returns JSON responses.

Task Description-5
Exception Handling Improvement.
Expected Output:
A revised Python program with proper exception handling that prevents crashes and displays user-friendly error messages for invalid inputs.
Prompt:
Convert the following Python searching algorithm into C language.Ensure the logic remains the same and provide a complete program with input and output.
CODE AND OUTPUT :

```python
def calculator():
    try:
        a = int(input("Enter first number: "))
        b = int(input("Enter second number: "))
        print("\nChoose operation:")
        print("1. Addition")
        print("2. Subtraction")
        print("3. Multiplication")
        print("4. Division")
        choice = int(input("Enter your choice (1-4): "))
        if choice == 1:
            print("Result:", a + b)
        elif choice == 2:
            print("Result:", a - b)
        elif choice == 3:
            print("Result:", a * b)
        elif choice == 4:
            print("Result:", a / b)
        else:
            print("Invalid choice! Please select between 1 and 4.")
    except ValueError:
        print("Invalid input! Please enter numbers only.")
    except ZeroDivisionError:
        print("Error! Division by zero is not allowed.")
    except Exception as e:
        print("Unexpected error:", e)
calculator()
```

```
Enter first number: 20
Enter second number: 20

Choose operation:
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice (1-4): 4
Result: 1.0
```

Explanation:
The original program could crash when users entered invalid data or attempted division by zero. The improved version uses try-except blocks to handle these errors

gracefully, preventing the program from stopping unexpectedly and providing clear, user-friendly error messages.